. Le changement de paradigme : Terminologie

Avant de toucher à l'interface, il faut traduire les concepts. Chez MongoDB, on ne parle plus de lignes ou de colonnes.

Concept SQL (MySQL)	Concept NoSQL (MongoDB)	Différence clé
Database	**Database**	Le conteneur principal reste identique.
Table	**Collection**	Une collection ne force pas une structure identique.
Row (Ligne)	**Document**	Un document est un objet JSON (BSON).
Column (Colonne)	**Field** (Champ)	On peut ajouter un champ à un document sans modifier les autres.
Join (Jointure)	**Embedding / Reference**	On préfère souvent imbriquer les données plutôt que de lier des tables.

2. L'Interface : MongoDB Compass

Pour les débutants, **MongoDB Compass** est l'équivalent de **phpMyAdmin**. C'est l'outil graphique indispensable.

Les étapes de création :

1. **Connexion** : On utilise une "Connection String" (ex: `mongodb://localhost:27017`).
2. **Bouton "Create Database"** : Contrairement à MySQL où l'on crée d'abord la base puis les tables, MongoDB demande souvent de créer **la base et la première collection simultanément**.
3. **L'onglet "Documents"** : C'est ici que l'on voit les données. On peut alterner entre une vue en liste (JSON) ou une vue en tableau.

3. Construction d'une structure (Schéma)

En SQL, on définit le type de chaque colonne avant d'insérer. En MongoDB, on insère un document, et la structure se crée "à la volée".

Exemple : Un utilisateur et ses commandes

En SQL, vous auriez deux tables et une clé étrangère. En MongoDB, on peut **imbriquer**.

Structure d'un document "User" :

```
{
  "_id": ObjectId("65a1b2c3d4e5"), // Généré automatiquement (clé primaire)
  "username": "JeanDupont",        // String
  "email": "jean@mail.com",
  "age": 28,                       // Number
  "active": true,                  // Boolean
  "address": {                     // Objet imbriqué (plus besoin de table 'adresses')
    "city": "Paris",
    "zip": "75000"
  },
  "roles": ["admin", "editor"]     // Array (très puissant en NoSQL)
}
```

4. Les fonctions de base (Le CRUD)

Pour manipuler ces données via un script (Node.js/Mongoose) ou dans la console Compass, voici les équivalences :

Créer (Insert)

- **MySQL** : `INSERT INTO users (username) VALUES ('Jean');`
- **MongoDB** : `db.users.insertOne({ username: "Jean" });`

Lire (Find)

- **MySQL** : `SELECT * FROM users WHERE age > 18;`
- **MongoDB** : `db.users.find({ age: { $gt: 18 } });` *(Note : `$gt` signifie "Greater Than")*

Mettre à jour (Update)

- **MySQL** : `UPDATE users SET active = true WHERE id = 1;`
- **MongoDB** : `db.users.updateOne({ _id: ... }, { $set: { active: true } });`

Supprimer (Delete)

- **MySQL** : `DELETE FROM users WHERE id = 1;`
- **MongoDB** : `db.users.deleteOne({ _id: ... });`

5. Conseil pédagogique : Le "Dénormalisation"

Le point le plus dur est d'accepter de **répéter de la donnée**.

- En SQL, on évite la duplication (Normalisation).
- En MongoDB, on favorise la **vitesse de lecture**.
  Si j'ai besoin du nom de l'auteur à chaque fois que j'affiche un article de blog, je stocke le nom de l'auteur *directement* dans le document de l'article au lieu de faire une jointure coûteuse.

L'exercice : Du Relationnel au Document

1. La vision SQL (Ce que l'on connaît)

En SQL, on sépare tout pour éviter la redondance.
On a besoin de deux tables et d'une **clé étrangère** pour lier les commentaires aux articles.

- **Table `posts`** : `id`, `title`, `content`
- **Table `comments`** : `id`, `post_id`, `author`, `text`

2. La vision MongoDB (Le changement)

Dans MongoDB, on se pose la question : "*A-t-on souvent besoin d'afficher les commentaires sans l'article ?*" La réponse est souvent **non**.
On va donc **imbriquer** les commentaires directement dans l'article.

Exemple de construction d'un document "Post" :

```
// On crée un seul objet qui contient TOUT

db.posts.insertOne({

  "title": "Apprendre MongoDB en DWWM",

  "content": "Le NoSQL est très flexible...",

  "author": "Formateur",

  "tags": ["Dev", "NoSQL", "BBD"], // Un tableau simple pour les catégories
```

```
  "comments": [ // Voici l'imbrication (Array d'objets)
    {
      "user": "Alice",
      "text": "Super article !",
      "date": new Date()
    },
    {
      "user": "Bob",
      "text": "Je préfère quand même MySQL...",
      "date": new Date()
    }
  ]
});
```

3. Mise en pratique sur MongoDB Compass

Pour ne pas être perdus dans l'interface, voici la procédure pas à pas :

1. **Création du "Container"** :
    - Cliquer sur le "**+**" à côté de "Databases".
    - Database Name : `blog_db`
    - Collection Name : `posts`
2. **Ajout de données (Le bouton "Add Data")** :
    - Choisir "Insert Document".
    - Nous pouvons copier-coller du JSON directement ou utiliser l'interface avec les petits "**+**" pour ajouter des champs.
3. **La Flexibilité** :
    - insérer un deuxième article, mais cette fois **sans le champ "tags"**.
    - *Leçon :* Contrairement à MySQL, MongoDB ne va pas râler.
      Chaque document est indépendant.

4. Les fonctions de requêtage (Le comparatif)

Voici un aide-mémoire qu'ils peuvent garder à côté d'eux :

Action	SQL	MongoDB (Mongoose/Shell)
Trouver un post	WHERE id = 1	findOne({ _id: ... })
Chercher un tag	WHERE tags LIKE '%Dev%'	find({ tags: "Dev" }) (très simple !)
Compter	SELECT COUNT(*) ...	countDocuments({})
Trier	ORDER BY date DESC	sort({ date: -1 })

Le "Point de vigilance"

Ce rappeler que **MongoDB est sensible à la casse**.

- `{ Author: "Jean" }` est différent de `{ author: "Jean" }`.
- En SQL, les noms de colonnes sont souvent insensibles selon la config, ici c'est du JavaScript pur !

📄 MÉMO : DE MYSQL À MONGODB (Spécial DWWM)

1. Vocabulaire : "Le traducteur"

Si tu penses (SQL)	Utilise (MongoDB)
Base de données	**Database**
Table	**Collection**
Ligne (Row)	**Document** (Format JSON/BSON)
Colonne (Column)	**Champ (Field)**
Jointure (Join)	**Imbrication (Embedding)** ou **Lookup**
ID (Auto-increment)	**_id** (Généré automatiquement : ObjectId)

2. Structure d'un Document (Le modèle)

Fini les colonnes figées ! Un document ressemble à un objet JavaScript.

JavaScript

```
{
  "_id": ObjectId("65b1..."),      // Clé primaire
automatique
  "titre": "Mon premier article",  // String
  "vues": 150,                     // Number (Int ou Float)
  "publie": true,                  // Boolean
  "tags": ["js", "nosql"],         // Array (Tableau)
  "auteur": {                      // Object (Imbrication)
    "nom": "Alice",
    "role": "Admin"
  }
}
```

3. Les commandes de survie (CRUD)

🟢 CREATE (Insérer)

- **Insérer un seul élément :** `db.ma_collection.insertOne({ nom: "Lucas", age: 22 });`
- **Insérer plusieurs éléments :** `db.ma_collection.insertMany([ {nom: "A"}, {nom: "B"} ]);`

🔵 READ (Lire / Chercher)

- **Tout récupérer :** `db.ma_collection.find();`
- **Chercher avec filtre (équivalent WHERE) :** `db.ma_collection.find({ nom: "Lucas" });`
- **Opérateurs de comparaison :**
  - `$gt` (Greater Than) : `db.ma_collection.find({ age: { $gt: 18 } });`
  - `$in` (Dans la liste) : `db.ma_collection.find({ ville: { $in: ["Paris", "Lyon"] } });`

🟡 UPDATE (Modifier)

Attention : Toujours utiliser `$set` sinon le document est remplacé entièrement !

- **Modifier un champ :** `db.ma_collection.updateOne({ _id: ... }, { $set: { age: 23 } });`
- **Ajouter un élément à un tableau :** `db.ma_collection.updateOne({ _id: ... }, { $push: { tags: "NouveauTag" } });`

🔴 DELETE (Supprimer)

- `db.ma_collection.deleteOne({ _id: ... });`
-

4. Astuces pour réussir son schéma

1. **Penser "Lecture" :** Si tu affiches toujours les commentaires avec l'article, mets les commentaires *dans* l'article (Array).
2. **Pas de schéma rigide :** Tu peux ajouter un champ `telephone` à un utilisateur sans devoir le mettre à tous les autres.
3. **L'ID est sacré :** L'identifiant se nomme toujours `_id`. En JavaScript (Node.js), on y accède via `document._id`.

🔌 CONNEXION & MODÈLES AVEC MONGOOSE

Mongoose permet de rajouter de la structure (un **Schema**) à la flexibilité de MongoDB pour éviter que n'importe quoi soit enregistré en base.

1. Initialisation de la connexion

Contrairement à `mysql2`, la connexion est souvent globale.

JavaScript

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/ma_base_dwwm')
  .then(() => console.log("Connexion réussie !"))
  .catch(err => console.error("Erreur de connexion", err));
```

2. Définir un Schéma (Le "Plan de construction")

C'est ici qu'on définit à quoi doit ressembler un document.

JavaScript

```
const userSchema = mongoose.Schema({
  username: { type: String, required: true, unique: true },
  email: String,
  age: Number,
  createdAt: { type: Date, default: Date.now }
});

// On transforme le schéma en Modèle (utilisable pour les
requêtes)
const User = mongoose.model('User', userSchema);
```

3. Les requêtes CRUD en JS (Async/Await)

C'est l'étape la plus concrète pour les apprenants. Rappelez-leur d'utiliser async/await.

Action	Code Mongoose
Créer	const newUser = await User.create({ username: "Leo" });
Lire tout	const users = await User.find();
Lire un seul	const user = await User.findById("ID_ICI");
Modifier	await User.updateOne({ _id: id }, { age: 25 });
Supprimer	await User.deleteOne({ _id: id });

💡 Le piège classique :

En SQL, si on oublie un champ, la base renvoie une erreur si la colonne est `NOT NULL`.
Avec Mongoose, si un champ n'est pas dans le **Schéma**, même si on l'envoie dans le `.create()`, il sera **ignoré** et ne sera pas enregistré en base.
Le Schéma est le "gardien" de la base.